Süzgeç – Digital Addiction Idea Proposal

Samsung Innovation Campus & Yeşilay Capstone Project

1. Project Idea

Problem. Driven by the fear of missing out (FoMO), users check their phones about 85 times a day, and more than half of these checks last less than 30 seconds (Andrews et al., 2015). Notifications feed this loop, while gambling and promotional notifications pull users toward addictive content; yet turning all notifications off increases anxiety and FoMO (Fitz et al., 2019).

Solution. Süzgeç (Turkish for "filter") is an Android app that keeps the phone in Do Not Disturb mode and sorts every notification into five categories based on its sender, app, content and time: Critical and Important notifications arrive immediately with sound, Normal and Unimportant ones are delivered as a summary three times a day at chosen times, and notifications related to the addiction type selected by the user (gambling, shopping promotions, social media) are deleted.

Target users. Android users aged 18 and over with non-flagship phones, primarily university students.

Specific goal. To reduce sound/vibration alerts by at least 50% and to reduce phone unlocks (from about 57 to 38 per day in Fitz et al.) while delivering at least 95% of Critical notifications, without increasing FoMO.

2. Literature Examples

Fitz et al. (2019), *Computers in Human Behavior*. In this randomized field experiment (237 participants), which is the main inspiration for Süzgeç, batching notifications three times a day reduced phone unlocks ($d = -0.60$), whereas receiving no notifications increased anxiety and FoMO. The lack of importance-based filtering, which the authors report as a limitation, is the gap that Süzgeç fills.

Visuri et al. (2019), *Int. J. Human-Computer Studies*; Lin et al. (2024), *Proc. ACM IMWUT*. In Visuri et al., an on-device decision tree trained on user labels decided whether to show or hide notifications with 91.1% accuracy. In Lin et al., generic automatic sorting was preferred in only 13.8% of comparisons, and users asked for a system that learns from their corrections.

3. Describe Your Data

Data	Source, format and size	Labels and use
Synthetic Turkish notifications	Self-generated (templates + LLM); JSONL; 5,000–10,000 samples	11 content types; main training data
Turkish SMS and e-mail datasets	TurkishSMSCollection, Uysal et al. (2013), UCI Turkish Spam; ~5,600 SMS + 826 e-mails	Spam/normal → content type; training
Team notifications	Real notifications from our team's phones (with consent, masked); JSONL	Double-labeled (kappa ≥ 0.70); test only
Open logs	Komninos et al. (2018, 2019), LSApp (2021); ~406k notifications, ~600k app-usage records	Unlabeled; descriptive analysis

Target variables and preprocessing. The model learns 11 user-independent content types (e.g., money transactions, promotions, gambling ads); each type is mapped to the 5 categories using the user's settings, and the order within the inbox is learned from the user's corrections. Service notifications are filtered out using Android flags and the 5-second rule, personal data is masked, Turkish-aware lowercasing is applied, and the test set is split by user and time.

4. Approach (Machine Learning or Deep Learning)

Choice: hybrid — deep learning for text, machine learning for context and personalization. Calls, alarms and priority apps are handled by a rule layer before the model.

- **Deep learning:** "2,500 TL has been withdrawn from your account" (Critical) and "Earn up to 2,500 TL in points on your card" (promotion) share words but differ in importance; to separate such texts, a small Turkish BERT model (turkish-mini-bert, 11.6M parameters) is fine-tuned and deployed on the phone. Since the data is limited, no model is trained from scratch.
- **Baseline and fallback model:** TF-IDF + logistic regression; if BERT cannot outperform it or exceeds 50 ms, the demo runs on this model.
- **Machine learning for context:** On app, sender and time data, classical methods perform as well as LSTMs (Katevas et al., 2017).
- **Personalization:** The user's category changes are applied on the device through a count-based correction; LightGBM LGBMRanker (NDCG@5) is tested for learning the user's reordering.